
Install Yatai

Welcome to Yatai! You will learn the system requirements, software dependencies, instructions for installation and how to migrate Yatai from a previous version.

System requirements

Yatai installation is supported on Linux-Unix based operating systems and Cloud platforms.

Hardware specifications

Before you begin, make sure you have:

- A Kubernetes cluster 1.20 or newer
- 4096 GB of memory
- 4 CPU

If you do not have a Kubernetes cluster in production, you can use [minikube](#).

Create your cluster with the command `minikube start --memory 4096 --cpus 4`.



Note: If you are using a Mac with minikube, using Docker as a driver is unsupported. To use minikube for installation:

1. Install

[hyperkit](#) with `brew install hyperkit`.

2. Use the flag

`--driver=hyperkit` when you start minikube. For example, `minikube start --driver=hyperkit`.

Software dependencies

- [Helm](#)

Quick install

If you want to quickly install a temporary Yatai cluster for trial or development purposes, you can use the following code that will have Yatai and its dependencies set up for you automatically.



Note: We advise that you do not use the following code. The following script installs MinIO, which is not as secure for production as S3 protocol storage. It is included for developmental testing.

```
DEVEL=true bash <(curl -s "https://raw.githubusercontent.com/bentoml/yatai/main/scripts/install-dev.sh")
```

Install and start Yatai

To install Yatai for production, you must do the following steps:

- **Step 1: Create a namespace**
- **Step 2: Connect to a database**
- **Step 3: Prepare storage**
- **Step 4: Install Yatai**

Step 1: Create a namespace

In your Kubernetes cluster, create a namespace for your Yatai.

If you are using:

- kubectl

```
kubectl create namespace yatai-system
```

- minikube

```
minikube kubectl -- create namespace yatai-system
```

You can use `minikube kubectl --` instead of `kubectl` for every command (See [minikube kubectl](#) for details).

Step 2: Connect to a database

You must use PostgreSQL or a PostgreSQL protocol database. For security, it is recommended that you use an RDS database instance.

Tab 1 - Use existing database

Connect the PostgreSQL database to your Kubernetes cluster. In the following example, a PostgreSQL database is used.

1. Connect to the database:

```
export PG_PASSWORD='your_password'
export PG_HOST=1.1.1.1
export PG_PORT=5432
export PG_DATABASE=yatai
export PG_USER=postgres
export PG_SSLMODE=disable

PGPASSWORD=$PG_PASSWORD psql \
  -h $PG_HOST \
  -p $PG_PORT \
  -U $PG_USER \
  -d postgres \
  -c "create database $PG_DATABASE"
```

2. After you connect to the database, test the connection of the database by using the following commands:

```
kubectl -n yatai-system delete pod postgresql-ha-client 2> /dev/null || true; \
kubectl run postgresql-ha-client --rm --tty -i --restart='Never' \
```

```
--namespace yatai-system \
--image docker.io/bitnami/postgresql-repmgr:14.4.0-debian-11-r13 \
--env="PGPASSWORD=$PG_PASSWORD" \
--command -- psql -h $PG_HOST -p $PG_PORT -U $PG_USER -d $PG_DA
```

If connection is successful, output is as follows:

```
?column?
-----
      1
(1 row)

pod "postgresql-ha-client" deleted
```

Tab 2- Create a new RDS database

You can configure and [create a database with the AWS management console](#) or create one from the command line. In this example, we create a PostgreSQL database instance.

Ensure that you have:

-  [jq](#)
- [AWS CLI](#)

1. Create the database.

```
export PG_PASSWORD=$(LC_ALL=C tr -dc 'A-Za-z0-9' < /dev/urandom | he
export PG_USER=yatai
export PG_DATABASE=yatai
export PG_SSLMODE=disable
export RDS_INSTANCE_IDENTIFIER=yatai-postgresql

aws rds create-db-instance \
  --db-name $PG_DATABASE \
  --db-instance-identifier $RDS_INSTANCE_IDENTIFIER \
```

```
--db-instance-class db.t3.micro \  
--engine postgres \  
--master-username $PG_USER \  
--master-user-password $PG_PASSWORD \  
--allocated-storage 20
```

2. Get the RDS instance and port.

```
read PG_HOST PG_PORT < <(echo $(aws rds describe-db-instances --db-in  
PG_HOST=$(sh -c "echo $PG_HOST")
```

3. Configure your bucket so that it has the correct permissions to accept inbound connection. See [Authorize inbound traffic for your Linux instances.](#)
4. After you set up the database, test the connection of the database by using the following commands:

```
kubectl -n yatai-system delete pod postgresql-ha-client 2> /dev/null || true; \  
kubectl run postgresql-ha-client --rm --tty -i --restart='Never' \  
--namespace yatai-system \  
--image docker.io/bitnami/postgresql-repmgr:14.4.0-debian-11-r13 \  
--env="PGPASSWORD=$PG_PASSWORD" \  
--command -- psql -h $PG_HOST -p $PG_PORT -U $PG_USER -d $PG_DA
```

If connection is successful, the table information is printed:

```
?column?
```

```
-----
```

```
1
```

```
(1 row)
```

```
pod "postgresql-ha-client" deleted
```

Step 3: Prepare storage

Configure the storage needed for your Yatai installation.

Tab 1 - Use an existing AWS S3 bucket

If you already have an S3 bucket, make sure your bucket credentials are saved as follows:

```
export S3_ENDPOINT=xxx
export S3_REGION=xxx
export S3_BUCKET_NAME=xxx
export S3_ACCESS_KEY=xxx
export S3_SECRET_KEY=xxx
export S3_SECURE=true
```

If you do not have access to a bucket, either create an S3 bucket or install MinIO.

Tab 2 - Create an AWS S3 bucket

Ensure that you have [AWS CLI](#).

1. Create an AWS bucket with the following commands:

```
export S3_BUCKET_NAME=yatai-registry
export S3_REGION=ap-northeast-3
export S3_ENDPOINT="s3.${S3_REGION}.amazonaws.com"
export S3_SECURE=true

aws s3api create-bucket \
  --bucket $S3_BUCKET_NAME \
  --region $S3_REGION \
  --create-bucket-configuration LocationConstraint=$S3_REGION
```

2. Retrieve your access key and export key. Store them in a secure location:

```
export S3_ACCESS_KEY=$(aws configure get default.aws_access_key_id)
export S3_SECRET_KEY=$(aws configure get default.aws_secret_access_key)
```

3. Verify your S3 connection:

```
kubectl -n yatai-system delete pod s3-client 2> /dev/null || true; \
kubectl run s3-client --rm --tty -i --restart='Never' \
  --namespace yatai-system \
  --env "AWS_ACCESS_KEY_ID=$S3_ACCESS_KEY" \
  --env "AWS_SECRET_ACCESS_KEY=$S3_SECRET_KEY" \
  --image quay.io/bentoml/s3-client:0.0.1 \
  --command -- sh -c "s3-client -e https://$S3_ENDPOINT listobj $S3_BUCKET"
```

If connection is successful, a success message is printed:

```
successfully
pod "s3-client" deleted
```

Tab 3 - Install MinIO



Note: Installing MinIO is not recommended since MinIO is not as secure for production as S3 protocol storage. It is included for developmental testing purposes.

1. Install the `minio-operator` helm chart.

```
helm repo add minio https://operator.min.io/
helm repo update minio
```

```
export S3_ACCESS_KEY=$(LC_ALL=C tr -dc 'A-Za-z0-9' < /dev/urandom | he
export S3_SECRET_KEY=$(LC_ALL=C tr -dc 'A-Za-z0-9' < /dev/urandom | he
```

```
cat <<EOF | helm upgrade --install minio-operator minio/minio-operator -n yatai-system
tenants:
- image:
  pullPolicy: IfNotPresent
  repository: quay.io/bentoml/minio-minio
  tag: RELEASE.2021-10-06T23-36-31Z
metrics:
  enabled: false
  port: 9000
mountPath: /export
name: yatai-minio
namespace: yatai-system
pools:
- servers: 4
  size: 20Gi
  volumesPerServer: 4
secrets:
  accessKey: $S3_ACCESS_KEY
  enabled: true
  name: yatai-minio
  secretKey: $S3_SECRET_KEY
  subPath: /data
EOF
```

2. Verify the `minio-operator` installation. Your installation is successful if the following command returns a status of Running or Completed.

```
kubectl -n yatai-system get pod -l app.kubernetes.io/name=minio-operator
```

NAME	READY	STATUS	RESTARTS	AGE
minio-operator-console-9d9cbbcc8-flzrw	1/1	Running	0	2m39s
minio-operator-6c984995c9-l8j2j	1/1	Running	0	2m39s

If the pods are not running immediately after installation, please wait a few minutes for them to start.

3. Verify the MinIO tenant installation:

```
kubectl -n yatai-system get pod -l app=minio
```

If verification is successful, the pod status is Running or Completed, as follows:

NAME	READY	STATUS	RESTARTS	AGE
yatai-minio-ss-0-0	1/1	Running	0	143m
yatai-minio-ss-0-1	1/1	Running	0	143m
yatai-minio-ss-0-2	1/1	Running	0	143m
yatai-minio-ss-0-3	1/1	Running	0	143m

4. Test your S3 connection.

```
export S3_ENDPOINT=minio.yatai-system.svc.cluster.local
export S3_REGION=foo
export S3_BUCKET_NAME=yatai
export S3_SECURE=false
export S3_ACCESS_KEY=$(kubectl -n yatai-system get secret yatai-minio -o
export S3_SECRET_KEY=$(kubectl -n yatai-system get secret yatai-minio -o

kubectl -n yatai-system delete pod s3-client 2> /dev/null || true; \
kubectl run s3-client --rm --tty -i --restart='Never' \
  --namespace yatai-system \
  --env "AWS_ACCESS_KEY_ID=$S3_ACCESS_KEY" \
  --env "AWS_SECRET_ACCESS_KEY=$S3_SECRET_KEY" \
  --image quay.io/bentoml/s3-client:0.0.1 \
  --command -- sh -c "s3-client -e http://$S3_ENDPOINT listbuckets && ech
```

If connection is successful, a success message is printed:

```
successfully
pod "s3-client" deleted
```

Step 4: Install Yatai

1. Install Yatai with helm charts and ensure the credentials with `$PG` are defined.

```
helm repo remove bentoml 2> /dev/null || true
helm repo add bentoml https://bentoml.github.io/helm-charts
helm repo update bentoml
helm upgrade --install yatai bentoml/yatai -n yatai-system \
  --set postgresql.host=$PG_HOST \
  --set postgresql.port=$PG_PORT \
  --set postgresql.user=$PG_USER \
  --set postgresql.database=$PG_DATABASE \
  --set postgresql.password=$PG_PASSWORD \
  --set postgresql.sslmode=$PG_SSLMODE \
  --set s3.endpoint=$S3_ENDPOINT \
  --set s3.region=$S3_REGION \
  --set s3.bucketName=$S3_BUCKET_NAME \
  --set s3.secure=$S3_SECURE \
  --set s3.accessKey=$S3_ACCESS_KEY \
  --set s3.secretKey=$S3_SECRET_KEY
```

2. Verify the Yatai installation:

```
kubectl -n yatai-system get pod -l app.kubernetes.io/name=yatai
```

If verification is successful, the pod status is Running or Completed, as follows:

NAME	READY	STATUS	RESTARTS	AGE
yatai-dbfbbb66f-67cq4	1/1	Running	0	45m

If the pods are not running immediately after installation, please wait a few minutes for them to start.

Congratulations on completing the install and welcome aboard to Yatai. After installation, you are the system administrator by default. You can easily designate another administrator or assign other user privileges.

Next steps

- Get hands-on with using Yatai in our tutorial.
- Ready to deploy your models? Check out .
- Install additional Observability components to improve model quality.

Uninstall

To uninstall Yatai from your Kubernetes cluster, download and run the [delete-yatai](#) bash script.



Note: This removes all Yatai resources, data, and cluster based storage in your Yatai namespace. External database and blob storage will not be affected.

Install Yatai deployment

Welcome to Yatai deployment! You will learn the system requirements, software dependencies, instructions for installing this Yatai add-on.

Before you begin, ensure that you have already installed [Yatai](#).

System requirements

Yatai deployment is an add-on to Yatai and has the same platform requirements and hardware specifications. See [Yatai hardware specifications](#).

Software dependencies

- [Helm](#)
- [Ingress controller](#)
 - *Optional:* If you are using minikube, ingress is included as an add-on. Simply enable ingress:

```
minikube addons enable ingress
```

- *Optional:* Registered DNS. By default, Yatai uses Magic DNS, which automatically registers DNS names for your network.

Quick Install



Note: We advise that you do not use the following code to install. It is included for testing purposes.

This script will automatically create the namespace `yatai-deployment` and install the following dependencies:

- cert-manager
- metrics-server
- docker-registry

```
DEVEL=true bash <(curl -s "https://raw.githubusercontent.com/bentoml/yatai-deployment/main/install.sh")
```

Install and start Yatai deployment

To install Yatai deployment for production, you must do the following steps:

Step 1: Create namespaces

In your Kubernetes cluster, create these namespaces for your Yatai installation.

If you are using:

- kubectl

```
# for yatai-deployment deployment \
kubectl create ns yatai-deployment \
# for bento image builder pods \
kubectl create ns yatai-builders \
```

```
# for bento deployment resources \
kubectl create ns yatai
```

- minikube

```
# for yatai-deployment deployment \
minikube kubectl -- create ns yatai-deployment \
# for bento image builder pods \
minikube kubectl -- create ns yatai-builders \
# for bento deployment resources \
minikube kubectl -- create ns yatai
```

You can use `minikube kubectl --` instead of `kubectl` for every command (See [minikube kubectl](#) for details).

Step 2: Install cert-manager

In this step, you install a self signed web certification for your Yatai cluster. If you have already installed cert-manager, [verify that it is running](#).

1. Install cert-manager:

```
kubectl apply -f https://github.com/cert-manager/cert-manager/releases/download
```

2. Verify the installation:

```
kubectl -n cert-manager get pod
```

If verification is successful, the pod status is Running or Completed, as follows:

NAME	READY	STATUS	RESTARTS	AGE
cert-manager-5dd59d9d9b-7js6w	1/1	Running	0	60s
cert-manager-cainjector-8696fc9f89-6grf8	1/1	Running	0	60s
cert-manager-webhook-7d4b5b8c56-7wrkf	1/1	Running	0	60s

If the pods are not running immediately after installation, please wait a few minutes for them to start.

3. Test if the webhook flow works with the following steps:

a. Create an Issuer for testing the webhook:

```
cat <<EOF > test-resources.yaml
apiVersion: v1
kind: Namespace
metadata:
  name: cert-manager-test
---
apiVersion: cert-manager.io/v1
kind: Issuer
metadata:
  name: test-selfsigned
  namespace: cert-manager-test
spec:
  selfSigned: {}
---
apiVersion: cert-manager.io/v1
kind: Certificate
metadata:
  name: selfsigned-cert
  namespace: cert-manager-test
spec:
  dnsNames:
    - example.com
  secretName: selfsigned-cert-tls
  issuerRef:
    name: test-selfsigned
EOF
```

b. Apply the issuer as a `yaml` file:

```
kubectl apply -f test-resources.yaml
```

c. Check the status of the certificate:

```
kubectl describe certificate -n cert-manager-test
```

You might wait for a few seconds as the cert-manager processes the request. If successful, the output is as follows:

```
...
Status:
  Conditions:
    Last Transition Time: 2022-08-12T09:11:03Z
    Message:             Certificate is up to date and has not expired
    Observed Generation: 1
    Reason:              Ready
    Status:              True
    Type:                Ready
  Not After:             2022-11-10T09:11:03Z
  Not Before:            2022-08-12T09:11:03Z
  Renewal Time:          2022-10-11T09:11:03Z
  Revision:              1
Events:
  Type    Reason    Age    From                                     Message
  ----    -
Normal Issuing    7s    cert-manager-certificates-trigger    Issuing ce
Normal Generated  6s    cert-manager-certificates-key-manager Sto
Normal Requested  6s    cert-manager-certificates-request-manager C
Normal Issuing    6s    cert-manager-certificates-issuing    The certi
```

d. Clean up the test resources.

```
kubectl delete -f test-resources.yaml
```

Step 3: Install metrics-server

To install metrics-server, run:

```
kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest
```

See [metrics-server](#) for more details.

Step 4: Install Docker registry

Do the following steps to install a new Docker registry.

1. Install the docker-registry helm chart:

```
helm repo add twuni https://helm.twun.io
helm repo update twuni
helm upgrade --install docker-registry twuni/docker-registry -n yatai-deployment
```

2. Verify the docker-registry installation:

```
kubectl -n yatai-deployment get pod -l app=docker-registry
```

3. If verification is successful, the pod status is Running or Completed, as follows:

NAME	READY	STATUS	RESTARTS	AGE
docker-registry-7dc8b575d4-d6stx	1/1	Running	0	10m

If the pod is not running immediately after installation, please wait a few minutes for them to start.

4. *Optional:* Create a Docker private registry proxy for development and testing. This allows you to build images locally and push them directly to a Kubernetes cluster.



Note: We advise that you do not use the following registry for production as it uses an insecure URL. It is included for testing purposes.

Configure your Docker registry with a prefix such as `127.0.0.1:5000/` .

```
cat <<EOF | kubectl apply -f -
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: docker-private-registry-proxy
  namespace: yatai-deployment
  labels:
    app: docker-private-registry-proxy
spec:
  selector:
    matchLabels:
      app: docker-private-registry-proxy
  template:
    metadata:
      creationTimestamp: null
    labels:
      app: docker-private-registry-proxy
  spec:
    containers:
      - args:
        - tcp
        - "5000"
        - docker-registry.yatai-deployment.svc.cluster.local
        image: quay.io/bentoml/proxy-to-service:v2
        name: tcp-proxy
        ports:
          - containerPort: 5000
            hostPort: 5000
```

```
name: tcp
protocol: TCP
resources:
  limits:
    cpu: 100m
    memory: 100Mi
EOF
```

5. Verify the private registry proxy:

```
kubectl -n yatai-deployment get pod -l app=docker-private-registry-proxy
```

If successful, the pod status is Running or Completed:

NAME	READY	STATUS	RESTARTS	AGE
docker-private-registry-proxy-jzjxr	1/1	Running	0	74s

If the pod is not running immediately after installation, please wait a few minutes for them to start.

When a Docker registry becomes available, prepare your connection parameters as follows:

```
export DOCKER_REGISTRY_SERVER=127.0.0.1:5000
export DOCKER_REGISTRY_IN_CLUSTER_SERVER=docker-registry.yatai-deployr
export DOCKER_REGISTRY_USERNAME=""
export DOCKER_REGISTRY_PASSWORD=""
export DOCKER_REGISTRY_SECURE=false
export DOCKER_REGISTRY_BENTO_REPOSITORY_NAME=yatai-bentos
```

Step 5: Configure your network

To configure network access for `BentoDeployment`, do the following steps:

1. Set an ingress class for `BentoDeployment` :

```
export INGRESS_CLASS=$(kubectl get ingressclass -o jsonpath='{.items[0].metadata.name}')
echo $INGRESS_CLASS
```

2. *Optional:* Configure your DNS. By default, Yatai uses [Magic DNS](#) to automatically generate the host for you.

If you choose to use a real DNS, do the following steps:

a. Map the external IP or CNAME from your DNS provider to your domain name with one of the following ways:

- If the Kubernetes networking layer (Load Balancer) uses an external IP address, then configure a wildcard record A as follows:

```
# Here yatai.example.com is the domain suffix for your cluster
*.yatai.example.com == A 35.233.41.212
```

- If the networking layer uses a CNAME, configure a CNAME record for the domain:

```
# Here yatai.example.com is the domain suffix for your cluster
*.yatai.example.com == CNAME a317a278525d111e89f272a164fd35fb
```

b. Configure Yatai to use your domain name:

```
export DOMAIN_SUFFIX=yatai.example.com
```

Step 6: Install Yatai deployment

Install the Yatai deployment add-on with the following steps:

1. Install the Yatai deployment helm chart:

```
helm repo remove bentoml 2> /dev/null || true
helm repo add bentoml https://bentoml.github.io/helm-charts
helm repo update bentoml
helm upgrade --install yatai-deployment bentoml/yatai-deployment -n yatai \
  --set dockerRegistry.server=$DOCKER_REGISTRY_SERVER \
```

```
--set dockerRegistry.inClusterServer=$DOCKER_REGISTRY_IN_CLUSTER \
--set dockerRegistry.username=$DOCKER_REGISTRY_USERNAME \
--set dockerRegistry.password=$DOCKER_REGISTRY_PASSWORD \
--set dockerRegistry.secure=$DOCKER_REGISTRY_SECURE \
--set dockerRegistry.bentoRepositoryName=$DOCKER_REGISTRY_BENTO_REPO_NAME \
--set layers.network.ingressClass=$INGRESS_CLASS
```

- a. *Optional:* You can add the flag `--set layers.network.ingressAnnotations.annotation1=annotation2` after `helm --install` to set annotations for the BentoDeployment ingress resource.

For example, if you want to set the ingress annotation "foo" to "bar", you can use the following code:

```
helm upgrade --install --set layers.network.ingressAnnotations.foo=bar \
--set dockerRegistry.server=$DOCKER_REGISTRY_SERVER \
--set dockerRegistry.inClusterServer=$DOCKER_REGISTRY_IN_CLUSTER \
--set dockerRegistry.username=$DOCKER_REGISTRY_USERNAME \
--set dockerRegistry.password=$DOCKER_REGISTRY_PASSWORD \
--set dockerRegistry.secure=$DOCKER_REGISTRY_SECURE \
--set dockerRegistry.bentoRepositoryName=$DOCKER_REGISTRY_BENTO_REPO_NAME \
--set layers.network.ingressClass=$INGRESS_CLASS
```

2. Verify the installation:

```
kubectl -n yatai-deployment get pod -l app.kubernetes.io/name=yatai-deployment
```

If successful, the pod status is Running or Completed:

NAME	READY	STATUS	RESTARTS	AGE
docker-private-registry-proxy-jzjxr	1/1	Running	0	74s

If the pod is not running immediately after installation, please wait a few minutes for them to start.

3. When your installation is complete and verified, configure your ingress class for Yatai deployment:

```
kubectl -n yatai-deployment patch cm/network --type merge --patch '{"c
```

4. *Optional:* If you set annotations when you installed Yatai deployment, you can configure it now:

```
kubectl -n yatai-deployment patch cm/network --type merge --patch '{"c
```

5. Verify that your domain is running by viewing the logs:

```
kubectl -n yatai-deployment logs -f job/yatai-deployment-default-domain
```

The logs will look similar to the following:

```
time="2022-08-16T14:48:11Z" level=info msg="Creating ingress default-  
time="2022-08-16T14:48:11Z" level=info msg="Waiting for ingress defau  
time="2022-08-16T14:48:41Z" level=info msg="Ingress default-domain-r  
time="2022-08-16T14:48:41Z" level=info msg="you have not set the don
```



Tip: To view your Yatai deployment installation logs, run:

```
kubectl -n yatai-deployment logs -f deploy/yatai-deployment
```

Congratulations on completing your Yatai deployment installation! If you're ready to deploy models, get a hands-on experience by reading our tutorial.

Observability (in progress)

After Yatai deployment is available and deploying models, you can use several tools to collect metrics and logs of your deployment status for model observability. As of now, Yatai supports collecting metrics and logs for deployment only and not Yatai itself.

Collecting metrics

Yatai uses [Prometheus](#) to track Bento deployment metrics. This section shows you step-by-step how to install Prometheus Stack for Yatai deployment.

Before you begin

Ensure that you have Yatai deployment. See [here](#) for details.

Quick install



Note: We advise that you do not use the following code. It is included for developmental testing.

The following code creates a namespace called `yatai-monitoring` and installs the required software dependencies.

```
bash <(curl -s "https://raw.githubusercontent.com/bentoml/yatai/main/scripts/qu
```

Install Prometheus

Migration



Note: Migrating data to Yatai 1.0 requires you to have Yatai version 0.46 or above.

Starting from version 1.0 of BentoML, components of Yatai are installed separately for smoother integration with a general DevOps ecosystem. Components such as deployment, logging, and monitoring features are now add-ons that you must install separately from the Yatai cluster.

The new installation method allows users to choose the features of Yatai that they need.

Prepare for upgrade

Your data and model files will not be affected if they are live in a stable, external platform. If your storage is on the same cluster as Yatai, you must back up and recover the data manually. This document will walk you through on how to back up your data. BentoML deployments will be completely unaffected and remain online.

Migrating to Yatai 1.0

To migrate, you must do the following steps:

- Step 1: Back up your data
- Step 2: Uninstall and re-install Yatai
- (Optional) Step 3: Install Yatai deployment

Step 1: Back up your data

This step guides you on backing up your database and object storage data. If you stored your data in an external database and/or blob storage, you do not have to back up PostgreSQL data and export object storage environment variables.

1. Back up PostgreSQL data with the following commands. This step must be skipped if you stored your data in an external database.

```
kill kubectl
kubectl port-forward --namespace yatai-system svc/yatai-postgresql 5433:5433
sleep 6
PGPASSWORD=$(kubectl -n yatai-system get secret yatai-postgresql -o jsonpath='{.data.pgpASSWORD}' | base64 --decode)
```

2. Get object storage environment variables with the following commands. This step must be skipped if you are using an external S3 bucket.

```
export S3_ENDPOINT=minio.yatai-components.svc.cluster.local
export S3_ACCESS_KEY=$(kubectl -n yatai-components get secret yatai-mir
export S3_SECRET_KEY=$(kubectl -n yatai-components get secret yatai-min
export S3_SECURE=false
export S3_BUCKET_NAME=yatai
export S3_REGION=i-dont-know
```

3. Test your object storage connection:

```
kubectl -n yatai-system delete pod s3-client 2> /dev/null || true; \
kubectl run s3-client --rm --tty -i --restart='Never' \
  --namespace yatai-system \
  --env "AWS_ACCESS_KEY_ID=$S3_ACCESS_KEY" \
  --env "AWS_SECRET_ACCESS_KEY=$S3_SECRET_KEY" \
  --image quay.io/bentoml/s3-client:0.0.1 \
  --command -- sh -c "s3-client -e http://$S3_ENDPOINT listbuckets && ech
```

If successful, a success message is printed:

```
successfully
pod "s3-client" deleted
```

Step 2: Uninstall and re-install Yatai

Uninstall Yatai from your Kubernetes cluster.

1. Download and run the [delete-yatai](#) bash script.
2. To reinstall Yatai, see [and do not install MinIO in Step 3: Prepare storage](#). Instead, install a new instance of PostgreSQL.
3. When PostgreSQL becomes available, run the following command to restore your data:


```
kill kubectl
kubectl port-forward --namespace yatai-system svc/postgresql-ha-pgpool 5
sleep 6
PGPASSWORD=$(kubectl -n yatai-system get secret postgresql-ha-postgres
```

(Optional) Step 3: Install Yatai deployment

After Yatai becomes available, you can reinstall Yatai deployment as an add-on if you want to deploy your models.

To reinstall Yatai deployment, see [and](#) do not install a new Docker registry in [Step 4: Install Docker registry](#). Instead, prepare your Docker connection parameters as follows:

```
export DOCKER_REGISTRY_SERVER=127.0.0.1:5000
export DOCKER_REGISTRY_IN_CLUSTER_SERVER=yatai-docker-registry.yatai-cc
export DOCKER_REGISTRY_USERNAME=""
export DOCKER_REGISTRY_PASSWORD=""
export DOCKER_REGISTRY_SECURE=false
export DOCKER_REGISTRY_BENTO_REPOSITORY_NAME=bentos
```

Troubleshoot

See these common installation questions for Yatai.

Pod stuck in pending status

If your pod is stuck in pending status after installing Yatai and unable to start, check if your firewalls, VPN, or other network configurations are blocking network access to your Kubernetes cluster.

Connection timeout to AWS service instance

If connecting to an AWS database or blob storage was unsuccessful, remember to make your AWS services publicly accessible and accept inbound connections

from your IP. By default, all AWS services are private. Please see [Permissions](#) for more details.

Limitations

Yatai won't run on Minikube with Docker as driver from MacOS

If your minikube is using Docker as a base image, it won't run Yatai successfully if run on a MacOS environment.

Workaround

Use hyperfix instead of Docker.

1. Install [hyperkit](#) with `brew install hyperkit` .

2. Use the flag

`--driver=hyperkit` when you start minikube. For example, `minikube start --driver=hyperkit` .

Applies to: Version 1.0 and older.

Parent topic:

[Untitled](#)

Learn more